

Ollama API Reference

API

| Note: Ollama's API docs are moving to <https://docs.ollama.com/api>

Endpoints

- [Generate a completion](#)
- [Generate a chat completion](#)
- [Create a Model](#)
- [List Local Models](#)
- [Show Model Information](#)
- [Copy a Model](#)
- [Delete a Model](#)
- [Pull a Model](#)
- [Push a Model](#)
- [Generate Embeddings](#)
- [List Running Models](#)
- [Version](#)
- [Experimental: Image Generation](#)

Conventions

Model names

Model names follow a `model:tag` format, where `model` can have an optional namespace such as `example/model`. Some examples are `orca-mini:3b-q8_0` and `llama3:70b`. The tag is optional and, if not provided, will default to `latest`. The tag is used to identify a specific version.

Durations

All durations are returned in nanoseconds.

Streaming responses

Certain endpoints stream responses as JSON objects. Streaming can be disabled by providing `{"stream": false}` for these endpoints.

Generate a completion

POST /api/generate

Generate a response for a given prompt with a provided model. This is a streaming endpoint, so there will be a series of responses. The final response object will include statistics and additional data from the request.

Parameters

- `model`: (required) the model name
- `prompt`: the prompt to generate a response for
- `suffix`: the text after the model response
- `images`: (optional) a list of base64-encoded images (for multimodal models such as `llava`)
- `think`: (for thinking models) should the model think before responding?

Advanced parameters (optional):

- `format`: the format to return a response in. Format can be `json` or a JSON schema
- `options`: additional model parameters listed in the documentation for the Model file such as `temperature`
- `system`: system message to (overrides what is defined in the Model file)
- `template`: the prompt template to use (overrides what is defined in the Model file)
- `stream`: if `false` the response will be returned as a single response object, rather than a stream of objects
- `raw`: if `true` no formatting will be applied to the prompt. You may choose to use the `raw` parameter if you are specifying a full templated prompt in your request to the API
- `keep_alive`: controls how long the model will stay loaded into memory following the request (default: 5m)
- `context` (deprecated): the context parameter returned from a previous request to `/generate`, this can be used to keep a short conversational memory

Experimental image generation parameters (for image generation models only):

[!WARNING] These parameters are experimental and may change in future versions.

- `width`: width of the generated image in pixels
- `height`: height of the generated image in pixels
- `steps`: number of diffusion steps

Structured outputs

Structured outputs are supported by providing a JSON schema in the `format` parameter. The model will generate a response that matches the schema. See the [structured outputs](#) example below.

JSON mode

Enable JSON mode by setting the `format` parameter to `json`. This will structure the response as a valid JSON object. See the JSON mode [example](#) below.

[!IMPORTANT] It's important to instruct the model to use JSON in the prompt. Otherwise, the model may generate large amounts of whitespace.

Examples

Generate request (Streaming)

Request

```
curl http://localhost:11434/api/generate -d '{
  "model": "llama3.2",
  "prompt": "Why is the sky blue?"
}'
```

Response

A stream of JSON objects is returned:

```
{
  "model": "llama3.2",
  "created_at": "2023-08-04T08:52:19.385406455-07:00",
  "response": "The",
  "done": false
}
```

The final response in the stream also includes additional data about the generation:

- `total_duration`: time spent generating the response
- `load_duration`: time spent in nanoseconds loading the model
- `prompt_eval_count`: number of tokens in the prompt
- `prompt_eval_duration`: time spent in nanoseconds evaluating the prompt
- `eval_count`: number of tokens in the response
- `eval_duration`: time in nanoseconds spent generating the response
- `context`: an encoding of the conversation used in this response, this can be sent in the next request to keep a conversational memory
- `response`: empty if the response was streamed, if not streamed, this will contain the full response

To calculate how fast the response is generated in tokens per second (token/s), divide `eval_count / eval_duration * 109`.

```
{
  "model": "llama3.2",
  "created_at": "2023-08-04T19:22:45.499127Z",
  "response": "",
  "done": true,
  "context": [1, 2, 3],
  "total_duration": 10706818083,
  "load_duration": 6338219291,
  "prompt_eval_count": 26,
  "prompt_eval_duration": 130079000,
  "eval_count": 259,
  "eval_duration": 4232710000
}
```

Request (No streaming)

Request

A response can be received in one reply when streaming is off.

```
curl http://localhost:11434/api/generate -d '{
  "model": "llama3.2",
  "prompt": "Why is the sky blue?",
  "stream": false
}'
```

Response

If stream is set to false, the response will be a single JSON object:

```
{
  "model": "llama3.2",
  "created_at": "2023-08-04T19:22:45.499127Z",
  "response": "The sky is blue because it is the color of the sky.",
  "done": true,
  "context": [1, 2, 3],
  "total_duration": 5043500667,
  "load_duration": 5025959,
  "prompt_eval_count": 26,
  "prompt_eval_duration": 325953000,
  "eval_count": 290,
  "eval_duration": 4709213000
}
```

Request (with suffix)

Request

```
curl http://localhost:11434/api/generate -d '{
  "model": "codellama:code",
  "prompt": "def fibonacci(n):",
  "suffix": "}"
```

```

"prompt": "def compute_gcd(a, b):",
"suffix": "    return result",
"options": {
    "temperature": 0
},
"stream": false
}'

```

Response

```

{
  "model": "codellama:code",
  "created_at": "2024-07-22T20:47:51.147561Z",
  "response": "\n  if a == 0:\n    return b\n  else:\n    return\ncompute_gcd(b % a, a)\n\ndef compute_lcm(a, b):\n  result = (a * b) / compute_gcd(a, b)\n",
  "done": true,
  "done_reason": "stop",
  "context": [...],
  "total_duration": 1162761250,
  "load_duration": 6683708,
  "prompt_eval_count": 17,
  "prompt_eval_duration": 201222000,
  "eval_count": 63,
  "eval_duration": 953997000
}

```

Request (Structured outputs)

Request

```

curl -X POST http://localhost:11434/api/generate -H "Content-Type: application/json" -d '{
  "model": "llama3.1:8b",
  "prompt": "Ollama is 22 years old and is busy saving the world. Respond using JSON",
  "stream": false,
  "format": {
    "type": "object",
    "properties": {
      "age": {
        "type": "integer"
      },
      "available": {
        "type": "boolean"
      }
    },
    "required": [
      "age",
      "available"
    ]
  }
}'

```

```
}  
'
```

Response

```
{  
  "model": "llama3.1:8b",  
  "created_at": "2024-12-06T00:48:09.983619Z",  
  "response": "{\n  \"age\": 22,\n  \"available\": true\n}",  
  "done": true,  
  "done_reason": "stop",  
  "context": [1, 2, 3],  
  "total_duration": 1075509083,  
  "load_duration": 567678166,  
  "prompt_eval_count": 28,  
  "prompt_eval_duration": 236000000,  
  "eval_count": 16,  
  "eval_duration": 269000000  
}
```

Request (JSON mode)

[!IMPORTANT] When format is set to json, the output will always be a well-formed JSON object. It's important to also instruct the model to respond in JSON.

Request

```
curl http://localhost:11434/api/generate -d '{  
  "model": "llama3.2",  
  "prompt": "What color is the sky at different times of the day?  
Respond using JSON",  
  "format": "json",  
  "stream": false  
}'
```

Response

```
{  
  "model": "llama3.2",  
  "created_at": "2023-11-09T21:07:55.186497Z",  
  "response": "{\n\n\"morning\": {\n\n\"color\": \"blue\"\n},  
  \n\n\"noon\": {\n\n\"color\": \"blue-gray\"\n},  
  \n\n\"afternoon\": {\n\n\"color\": \"warm gray\"\n},  
  \n\n\"evening\": {\n\n\"color\": \"orange\"\n}\n}\n",  
  "done": true,  
  "context": [1, 2, 3],  
  "total_duration": 4648158584,  
  "load_duration": 4071084,  
  "prompt_eval_count": 36,  
  "prompt_eval_duration": 439038000,
```

```

    "eval_count": 180,
    "eval_duration": 4196918000
}

```

The value of response will be a string containing JSON similar to:

```

{
  "morning": {
    "color": "blue"
  },
  "noon": {
    "color": "blue-gray"
  },
  "afternoon": {
    "color": "warm gray"
  },
  "evening": {
    "color": "orange"
  }
}

```

Request (with images)

To submit images to multimodal models such as llava or bakllava, provide a list of base64-encoded images:

Request

```

curl http://localhost:11434/api/generate -d '{
  "model": "llava",
  "prompt": "What is in this picture?",
  "stream": false,
  "images":
["iVBORw0KGgoAAAANSUhEUgAAAG0AAABmCAYAAADBPx+VAAAACXBIWMAAAAsTAAALF
7zZo888yz1r7MnDl7z5xvsjkzs2fP3uu71nNfa7lkAsm7d+
+Sffv2JbNmzUqcc8m0ad0SszZs3Z+/
XES4ZckAWJEGWPiCxjsQNLWmQsWjRIpMseaxcuTKpG/7HP27I8P79e7dq1ars/yL4/
v27S0ejqvw+cU0GEGGpKHR37tzJCEpHV9tnT58+dXXCJDdECBE20jr qjh071hpNECj>
+pyRkRFXKyRDAqxEp4yMLDDzXG1NPnnyJKkThoK0VFd1ELZu3TrzXKxKfW7dMBQ6bcu
6DZELQd/09C9QGq5RsmSRybqkwHGjh070sJSsYYm3ijPpyHzoiaacg35MLdDSIS/
01yM778j0TwYUkKNHWUzUWa0syle00MyI0fcn0wIdjvtNdW/
HZwNLGg+sR1kMepSNJXmIwxBZiG8tDTpEzZKg0GItns0sY8USkxDhD0Rinuiko2gfL,
RbiD2LZAJU9zKQJj8RDR0vJBR1/
Phx9+PHj9Z7REF4nTZkxzX4LCXHrV271qXkBAPGFp/atWvu/
PnzHe4C97F48eIsRLZ9+3a3f/
9+87dwP1JxaF7/3r17ba+5l4EcaVo0lj3SBq5kGTJSQmLWMjgYNei2GPT1MuMqGTDEI
jGnkmPJ/nhccs44jvDAXpVcxnq0F6eT8h4ni/
iIWpR5lPyA6ETkNXoSukvpJAD3AsXLiwpZs49+fPn5ke4j10TqYvegSfn00nafC+Tv<
JPkgQysqQNBzagXY55n0/
oa1F7qvIPWkRL12WRpMWUvpVDYmxAPehxWSe8ZEXL20sadYIozfmNch4QJPAfeJgW3i
oJU4eiEP0jVoAnHa9wyJycITMP78+eMeP37sXrx44d6+fdt6f82aNdKx1pg9e3Zb5W-
PiAP//
vuvW7BggZszZ072/+TJk91YgkafPn166zXB1rQHFvouAWHq9z3SEevSUerqCn2/

```

dDCeta2jxYbr69evk4MHDyY7d+7MjhMnTiTPnz9Pfv/
+nfQT2ggp02dMF8cghuoM7Ygj5iWCqRlGFml0QC/
ftGmTmzt3rmsaKDsGBSP0/8yPeLLBihLk0KJc0jp8H8vUzcxIA1k6QJ/
c78tWEyj5P3o4u9+jyWNPdJi5rAH9x0KHcl4Hg570eQp3+vHXGyrmEeigzQsQsjavXt
p49e1qjEP2mwTViNRo0VJWH1deMXcNK08uUjVUu7s/
zRaL+oLNxz1bpANco4npUgX4G2eFbpDFyQoQxojBCpEGSytm0H8qrH5Q9vuzD6ofQy
vQi1T+rJj7WiTz4Pt/l3LxUkr5P2VYZaZ4URpsE+st/dujQoaBBYokbrz/
8TJNQYLSonrPS9kUaSkPeZyj1AWSj+d+VBoy1pIwVNed8P0Ll/
ee5HdGRhrHhR5GGN0r4LGZBaj8oFDJitBTJzIZgFcmU0Y8ytWMZmZJ0aXUSrUs5RxKr
nz9/
zpxnwLES0Ndg6dKlbsaMGS4EHFhtjFIDHwK0o46l4TxSuxgDzi+rE2jg+BaFru0X4H
r6zD97WK2qFnGjBxTw5qNGPxT+5T/
r7/7RawFC3j4vTp09koCckeHjqbHJqArmH5UrFKKksnxrk7FuRIs8STfBZv+luugXZ
pP90is4z+TiMzUUKUjD0iEi1fzX8GmXyuxUBRcaUfykV0YZnlJGKQp0iGB76x5GeWk
l7emTLHi7vwTdirNEt0qxnzAvBFcnQF16xh/
TMpUuXHDowhlA9vQVraQhkudRdzOnK+04ZSP3DUhVSP61YsaLtd/
ks7ZgtPcXqPqEafHkdqa84X6aCeL7Ywlv6edGFHb+ZFIcPlLjHhg0bKuk0CSvVznWsc
v7Ab8N05/9t27Z16KUqoFGsxnI9EosS2niSYg9SpU6B4JgTrvVW1flt1sT+0ADIJU2r
AcsgNyL74DQ0730sv78Su7+t/
A36MdY0sw5o40ahsLXr58aZ5HtZB8GH64m9EmMZ7FpYw4T6QnrZfgenrhFxaSiSGXtf
jhRDUq8MKrTho9fEkihMmhxtBI1DxKFY9XLpVcSkfoi8JGnToZ05sU5aiDQIW716dd
jBmggIrXJWlZM6pSETsEPGqZ0ndr2uuuR5rF169a2HoHPdurUKZM4C01WTPqaDaAd+(
lN5rZubMmR2bE4SIC4nJoltgAV/
dVefZm72AtctUCJU2CMJ327hxY9t7EHbkyJFseq+EJSY16RPo3Dkq1kk7+q0bNmyD
oSoEM0k2CQGl4ckhBwaetBhjCwH0ZhtJR0PJkyc7UjcyLDjmrH7ADTEBXXf0YmB0k
PW5YMHD9yZM2faPsMwoc7RciYJXbGuBqJ1UIGKKLv915jsvgtJxCZDubdXr165mzdvt
TzXyAKV0mmHWFVsbDNw1l1+4f90U6IY/q4V27dpnE9bJ+v87QEYdjqx/
UamVVRG+mwkNTYN+9tjkwzEx+atCm/
X9WvWtDtAb68Wy9LXa1UmvCDDIpPky0Q5ZwSzJ4jMrvFcr0rSj0Uh+GcT4LSg5ugkW
SCDQBojh0hPlaJdah+tkVYrnTZowP8iq1F1TgMBBauufyB33x1v+NWFYmT5KmppgHC
UivS9NztptkVEDkCrs5alhhWP9NeqlfWopzhZScI6QxseegZRGeg5a8C3Re1Mfl1ScP
FC3mp6jBhKK0iY2h3gtUV64TWM6wDETRPLDfSakXmH3w8g9Jlug8ZtTt4kVF0kLUYYr
DrQ5YhMGBa9L3ucdjh0y8k0HW5gU/VEEmJTcL4Pz/
f7mgoAbYkAAAAAElFTkSuQmCC"]
}'

Response

```
{  
  "model": "llava",  
  "created_at": "2023-11-03T15:36:02.583064Z",  
  "response": "A happy cartoon character, which is cute and  
              cheerful.",  
  "done": true,  
  "context": [1, 2, 3],  
  "total_duration": 2938432250,  
  "load_duration": 2559292,  
  "prompt_eval_count": 1,  
  "prompt_eval_duration": 2195557000,  
  "eval_count": 44,  
  "eval_duration": 736432000  
}
```


Request (Raw Mode)

In some cases, you may wish to bypass the templating system and provide a full prompt. In this case, you can use the `raw` parameter to disable templating. Also note that raw mode will not return a context.

Request

```
curl http://localhost:11434/api/generate -d '{
  "model": "mistral",
  "prompt": "[INST] why is the sky blue? [/INST]",
  "raw": true,
  "stream": false
}'
```

Request (Reproducible outputs)

For reproducible outputs, set `seed` to a number:

Request

```
curl http://localhost:11434/api/generate -d '{
  "model": "mistral",
  "prompt": "Why is the sky blue?",
  "options": {
    "seed": 123
  }
}'
```

Response

```
{
  "model": "mistral",
  "created_at": "2023-11-03T15:36:02.583064Z",
  "response": " The sky appears blue because of a phenomenon
              called Rayleigh scattering.",
  "done": true,
  "total_duration": 8493852375,
  "load_duration": 6589624375,
  "prompt_eval_count": 14,
  "prompt_eval_duration": 119039000,
  "eval_count": 110,
  "eval_duration": 1779061000
}
```

Generate request (With options)

If you want to set custom options for the model at runtime rather than in the Modelfile, you can do so with the options parameter. This example sets every available option, but you can set any of them individually and omit the ones you do not want to override.

Request

```
curl http://localhost:11434/api/generate -d '{
  "model": "llama3.2",
  "prompt": "Why is the sky blue?",
  "stream": false,
  "options": {
    "num_keep": 5,
    "seed": 42,
    "num_predict": 100,
    "top_k": 20,
    "top_p": 0.9,
    "min_p": 0.0,
    "typical_p": 0.7,
    "repeat_last_n": 33,
    "temperature": 0.8,
    "repeat_penalty": 1.2,
    "presence_penalty": 1.5,
    "frequency_penalty": 1.0,
    "penalize_newline": true,
    "stop": ["\n", "user:"],
    "numa": false,
    "num_ctx": 1024,
    "num_batch": 2,
    "num_gpu": 1,
    "main_gpu": 0,
    "use_mmap": true,
    "num_thread": 8
  }
}'
```

Response

```
{
  "model": "llama3.2",
  "created_at": "2023-08-04T19:22:45.499127Z",
  "response": "The sky is blue because it is the color of the sky.",
  "done": true,
  "context": [1, 2, 3],
  "total_duration": 4935886791,
  "load_duration": 534986708,
  "prompt_eval_count": 26,
  "prompt_eval_duration": 107345000,
  "eval_count": 237,
```

```
    "eval_duration": 4289432000
}
```

Load a model

If an empty prompt is provided, the model will be loaded into memory.

Request

```
curl http://localhost:11434/api/generate -d '{
  "model": "llama3.2"
}'
```

Response

A single JSON object is returned:

```
{
  "model": "llama3.2",
  "created_at": "2023-12-18T19:52:07.071755Z",
  "response": "",
  "done": true
}
```

Unload a model

If an empty prompt is provided and the `keep_alive` parameter is set to `0`, a model will be unloaded from memory.

Request

```
curl http://localhost:11434/api/generate -d '{
  "model": "llama3.2",
  "keep_alive": 0
}'
```

Response

A single JSON object is returned:

```
{
  "model": "llama3.2",
  "created_at": "2024-09-12T03:54:03.516566Z",
  "response": "",
  "done": true,
  "done_reason": "unload"
}
```

Generate a chat completion

POST /api/chat

Generate the next message in a chat with a provided model. This is a streaming endpoint, so there will be a series of responses. Streaming can be disabled using "stream": false. The final response object will include statistics and additional data from the request.

Parameters

- model: (required) the model name
- messages: the messages of the chat, this can be used to keep a chat memory
- tools: list of tools in JSON for the model to use if supported
- think: (for thinking models) should the model think before responding?

The message object has the following fields:

- role: the role of the message, either system, user, assistant, or tool
- content: the content of the message
- thinking: (for thinking models) the model's thinking process
- images (optional): a list of images to include in the message (for multimodal models such as llava)
- tool_calls (optional): a list of tools in JSON that the model wants to use
- tool_name (optional): add the name of the tool that was executed to inform the model of the result

Advanced parameters (optional):

- format: the format to return a response in. Format can be json or a JSON schema.
- options: additional model parameters listed in the documentation for the Modelfile such as temperature
- stream: if false the response will be returned as a single response object, rather than a stream of objects
- keep_alive: controls how long the model will stay loaded into memory following the request (default: 5m)

Tool calling

Tool calling is supported by providing a list of tools in the tools parameter. The model will generate a response that includes a list of tool calls. See the Chat request (Streaming with tools) example below.

Models can also explain the result of the tool call in the response. See the Chat request (With history, with tools) example below.

See models with tool calling capabilities.

Structured outputs

Structured outputs are supported by providing a JSON schema in the format parameter. The model will generate a response that matches the schema. See the [Chat request \(Structured outputs\)](#) example below.

Examples

Chat request (Streaming)

Request

Send a chat message with a streaming response.

```
curl http://localhost:11434/api/chat -d '{
  "model": "llama3.2",
  "messages": [
    {
      "role": "user",
      "content": "why is the sky blue?"
    }
  ]
}'
```

Response

A stream of JSON objects is returned:

```
{
  "model": "llama3.2",
  "created_at": "2023-08-04T08:52:19.385406455-07:00",
  "message": {
    "role": "assistant",
    "content": "The",
    "images": null
  },
  "done": false
}
```

Final response:

```
{
  "model": "llama3.2",
  "created_at": "2023-08-04T19:22:45.499127Z",
  "message": {
    "role": "assistant",
    "content": ""
  },
  "done": true,
  "total_duration": 4883583458,
  "load_duration": 1334875,
```

```
"prompt_eval_count": 26,  
"prompt_eval_duration": 342546000,  
"eval_count": 282,  
"eval_duration": 4535599000  
}
```

Chat request (Streaming with tools)

Request

```
curl http://localhost:11434/api/chat -d '{  
  "model": "llama3.2",  
  "messages": [  
    {  
      "role": "user",  
      "content": "what is the weather in tokyo?"  
    }  
  ],  
  "tools": [  
    {  
      "type": "function",  
      "function": {  
        "name": "get_weather",  
        "description": "Get the weather in a given city",  
        "parameters": {  
          "type": "object",  
          "properties": {  
            "city": {  
              "type": "string",  
              "description": "The city to get the weather for"  
            }  
          },  
          "required": ["city"]  
        }  
      },  
      "required": ["city"]  
    }  
  ],  
  "stream": true  
}'
```

Response

A stream of JSON objects is returned:

```
{  
  "model": "llama3.2",  
  "created_at": "2025-07-07T20:22:19.184789Z",  
  "message": {  
    "role": "assistant",  
    "content": "",  
    "tool_calls": [  
      {  
        "function": {  
          "name": "get_weather",  
          "arguments": {  
            "city": "tokyo"  
          }  
        }  
      }  
    ]  
  }  
}
```

```

        "function": {
            "name": "get_weather",
            "arguments": {
                "city": "Tokyo"
            }
        }
    },
    "done": false
}

```

Final response:

```

{
  "model": "llama3.2",
  "created_at": "2025-07-07T20:22:19.19314Z",
  "message": {
    "role": "assistant",
    "content": ""
  },
  "done_reason": "stop",
  "done": true,
  "total_duration": 182242375,
  "load_duration": 41295167,
  "prompt_eval_count": 169,
  "prompt_eval_duration": 24573166,
  "eval_count": 15,
  "eval_duration": 115959084
}

```

Chat request (No streaming)

Request

```

curl http://localhost:11434/api/chat -d '{
  "model": "llama3.2",
  "messages": [
    {
      "role": "user",
      "content": "why is the sky blue?"
    }
  ],
  "stream": false
}'

```

Response

```

{
  "model": "llama3.2",
  "created_at": "2023-12-12T14:13:43.416799Z",
  "message": {

```

```

    "role": "assistant",
    "content": "Hello! How are you today?"
  },
  "done": true,
  "total_duration": 5191566416,
  "load_duration": 2154458,
  "prompt_eval_count": 26,
  "prompt_eval_duration": 383809000,
  "eval_count": 298,
  "eval_duration": 4799921000
}

```

Chat request (No streaming, with tools)

Request

```

curl http://localhost:11434/api/chat -d '{
  "model": "llama3.2",
  "messages": [
    {
      "role": "user",
      "content": "what is the weather in tokyo?"
    }
  ],
  "tools": [
    {
      "type": "function",
      "function": {
        "name": "get_weather",
        "description": "Get the weather in a given city",
        "parameters": {
          "type": "object",
          "properties": {
            "city": {
              "type": "string",
              "description": "The city to get the weather for"
            }
          }
        },
        "required": ["city"]
      }
    }
  ],
  "stream": false
}'

```

Response

```

{
  "model": "llama3.2",
  "created_at": "2025-07-07T20:32:53.844124Z",
  "message": {

```



```

    "role": "assistant",
    "content": "",
    "tool_calls": [
      {
        "function": {
          "name": "get_weather",
          "arguments": {
            "city": "Tokyo"
          }
        }
      }
    ]
  },
  "done_reason": "stop",
  "done": true,
  "total_duration": 3244883583,
  "load_duration": 2969184542,
  "prompt_eval_count": 169,
  "prompt_eval_duration": 141656333,
  "eval_count": 18,
  "eval_duration": 133293625
}

```

Chat request (Structured outputs)

Request

```

curl -X POST http://localhost:11434/api/chat -H "Content-Type:
application/json" -d '{
  "model": "llama3.1",
  "messages": [{"role": "user", "content": "Ollama is 22 years old
and busy saving the world. Return a JSON object with the age and
availability."}],
  "stream": false,
  "format": {
    "type": "object",
    "properties": {
      "age": {
        "type": "integer"
      },
      "available": {
        "type": "boolean"
      }
    }
  },
  "required": [
    "age",
    "available"
  ]
},
"options": {
  "temperature": 0
}

```

```
}  
}'
```

Response

```
{  
  "model": "llama3.1",  
  "created_at": "2024-12-06T00:46:58.265747Z",  
  "message": {  
    "role": "assistant",  
    "content": "{\\"age\\": 22, \\"available\\": false}"  
  },  
  "done_reason": "stop",  
  "done": true,  
  "total_duration": 2254970291,  
  "load_duration": 574751416,  
  "prompt_eval_count": 34,  
  "prompt_eval_duration": 1502000000,  
  "eval_count": 12,  
  "eval_duration": 175000000  
}
```

Chat request (With History)

Send a chat message with a conversation history. You can use this same approach to start the conversation using multi-shot or chain-of-thought prompting.

Request

```
curl http://localhost:11434/api/chat -d '{  
  "model": "llama3.2",  
  "messages": [  
    {  
      "role": "user",  
      "content": "why is the sky blue?"  
    },  
    {  
      "role": "assistant",  
      "content": "due to rayleigh scattering."  
    },  
    {  
      "role": "user",  
      "content": "how is that different than mie scattering?"  
    }  
  ]  
}'
```

Response

A stream of JSON objects is returned:

```
{
  "model": "llama3.2",
  "created_at": "2023-08-04T08:52:19.385406455-07:00",
  "message": {
    "role": "assistant",
    "content": "The"
  },
  "done": false
}
```

Final response:

```
{
  "model": "llama3.2",
  "created_at": "2023-08-04T19:22:45.499127Z",
  "done": true,
  "total_duration": 8113331500,
  "load_duration": 6396458,
  "prompt_eval_count": 61,
  "prompt_eval_duration": 398801000,
  "eval_count": 468,
  "eval_duration": 7701267000
}
```

Chat request (With history, with tools)

Request

```
curl http://localhost:11434/api/chat -d '{
  "model": "llama3.2",
  "messages": [
    {
      "role": "user",
      "content": "what is the weather in Toronto?"
    },
    // the message from the model appended to history
    {
      "role": "assistant",
      "content": "",
      "tool_calls": [
        {
          "function": {
            "name": "get_weather",
            "arguments": {
              "city": "Toronto"
            }
          }
        }
      ]
    }
  ],
  // the tool call result appended to history
  {
```

```

        "role": "tool",
        "content": "11 degrees celsius",
        "tool_name": "get_weather"
    }
],
"stream": false,
"tools": [
    {
        "type": "function",
        "function": {
            "name": "get_weather",
            "description": "Get the weather in a given city",
            "parameters": {
                "type": "object",
                "properties": {
                    "city": {
                        "type": "string",
                        "description": "The city to get the weather for"
                    }
                }
            },
            "required": ["city"]
        }
    }
]
}'

```

Response

```

{
  "model": "llama3.2",
  "created_at": "2025-07-07T20:43:37.688511Z",
  "message": {
    "role": "assistant",
    "content": "The current temperature in Toronto is 11°C."
  },
  "done_reason": "stop",
  "done": true,
  "total_duration": 890771750,
  "load_duration": 707634750,
  "prompt_eval_count": 94,
  "prompt_eval_duration": 91703208,
  "eval_count": 11,
  "eval_duration": 90282125
}

```

Chat request (with images)

Request

Send a chat message with images. The images should be provided as an array, with the individual images encoded in Base64.

```
curl http://localhost:11434/api/chat -d '{
  "model": "llava",
  "messages": [
    {
      "role": "user",
      "content": "what is in this image?",
      "images":
["iVBORw0KGgoAAAANSUhEUgAAAG0AAABmCAYAAADBPx+VAAAACXBIWXMAAAAsTAAALi
7zZo888yz1r7MnDl7z5xvsjkzs2fP3uu71nNfa7lkAsm7d+
+Sffv2JbNmzUqcc8m0ad0SzZs3Z+/
XES4ZckAWJEGWPiCxjsQNLWmQsWjRIpMseaxcuTKpG/7HP27I8P79e7dq1ars/yL4/
v27S0ejqvw+cU0GEGGpKHR37tzJCEpHV9tnT58+dXXCJDdECBE20jrqrjh071hpNECj>
+pyRkRFXKyRDAqxEp4yMLDDzXG1NPnnYJKkThoK0VFd1ELZu3TrzXKxKfW7dMBQ6bci
6DZELQd/09C9QGq5RsmSRybqkwHGjh070sJSsYYm3ijPpyHzoiacg35MLdDSIS/
01yM778j0TWyUkKNHWUzUWa0syle00MyI0fcn0wIdjvtNdW/
HZwNLGg+sR1kMepSNJXmIwxBZiG8tDTpEzZKg0GItns0sY8USkxDhD0Rinuiko2gfl,
RbiD2LZAJU9zKQJj8RDR0vJBR1/
Phx9+PHj9Z7REF4nTZkxzX4LCXHrV271qXkBAPGFp/atWvu/
PnzHe4C97F48eIsRLZ9+3a3f/
9+87dwP1JxaF7/3r17ba+5l4EcaVo0lj3SBq5kGTJSQmLWMjgYNei2GPT1MuMqGTDEI
jGnmPJ/nhccs44jvDAXpVcxnq0F6eT8h4ni/
iIWpR5lPyA6ETkNXoSukvpJAD3AsXLiwpZs49+fPn5ke4j10TqYvegSfn0nafC+Tv?
JPkgQysqQNBzagXY55n0/
oa1F7qvIPWkRL12WRpMWUvpVDYmxAPehxWSe8ZEXL20sadYIozfmNch4QJPAfeJgW3i
oJU4eiEP0jVoAnHa9wyJycITMP78+eMeP37sXrx44d6+fdt6f82aNdKx1pg9e3Zb5W-
PiAP//
vuvW7BggZszZ072/+TJk91YgkafPn166zXB1rQHfvouAWHq9z3SEevSUerqCn2/
dDCeta2jxYbr69evk4MHDyY7d+7MjhMnTiTPnz9Pfv/
+nfQT2ggp02dMF8cghuoM7Ygj5iWCqRLGFml0QC/
ftGmTmzt3rmsaKDsgBSP0/8yPeLLBiHk0KJc0jp8H8vUzcxIA1k6QJ/
c78tWEYj5P3o4u9+jywNPdJi5rAH9x0KHcl4Hg570eQp3+vHXGyrmEeigzQsQsjavXi
p49e1qjEP2mwTViNRo0VJWH1deMXcNK08uUjVUu7s/
zRaL+oLNxz1bpANco4npUgX4G2eFbpDFyQoXojBCpEGSytm0H8qrH5Q9vuzD6ofQyI
vQi1T+rJj7WiTz4Pt/l3LxUkr5P2VYZaZ4URpsE+st/dujQoaBBYokbrz/
8TJNQYLSonrPS9kUaSkPeZyj1AWSj+d+VBoy1pIWNed8P0Ll/
ee5HdGRhrHhR5GGN0r4LGZBaj8oFDJitBTJzIZgFcmU0Y8ytWMZMzJ0aXUSrUs5RxKr
nz9/
zpxnwLES0ndg6dKlbsaMGS4EHFhtjFIDHwK0o46l4TxSuxgDzi+rE2jg+BaFru0X4H)
r6zD97WK2qFnGjBxTw5qNGPxT+5T/
r7/7RawFC3j4vTp09koCckeHjqbHJqArmH5UrFKKksnxrK7FuRIs8STfBZv+luugXZ:
pP90is4z+TiMzUUKUjD0iEi1fzX8GmXyuxUBRcaUfykV0YZnlJGKQp0iGB76x5Gekw
l7emTLHi7vwTdirNEt0qxnzAvBFcnQF16xh/
TMpUuXHDowhLA9vQVraQhkudRdz0nK+04ZSP3DUhVSP61YsaLtd/
ks7ZgtPcXqPqEafHkdqa84X6aCeL7Ywlv6edGFHb+ZFICPl1jHhg0bKuk0CSvVznWsc
v7Ab8N05/9t27Z16KUqoFGsxnI9EosS2niSYg9SpU6B4JgTrvVW1fl1t1st+0ADIJU2r
AcsgNyL74DQq730sv78Su7+t/
```

```

A36MdY0sW5o40ahsLXr58aZ5HtZB8GH64m9EmMZ7FpYw4T6QnrZfgenrhFxaSiSGXtf
jhRDUq8MKrTho9fEkihMmhxtBI1DxKFY9XLpVcSkfoi8JGnToZ05sU5aiDQIW716ddt
jBmggIrXJWlZM6pSETsEPGqZ0ndr2uuuR5rF169a2HoHPdurUKZM4C01WTPqaDaAd+(
lN5rZubMmR2bE4SIC4nJoltgAV/
dVefZm72AtctUCJU2CMJ327hxY9t7EHbkyJFseq+EJSY16RPo3Dkq1kkr7+q0bNmyDu
oSoEM0k2CQGlR4ckhBwaetBhjCwH0ZhtJR0PJkyc7UjcYLDjmrH7ADTEBXf0YmB0k9
PW5YMHD9yZM2faPsMwoc7RciYJXbGuBqJ1UIGKKLv915jsvgtJxCZDubdXr165mzdvt
TzXyAKV0mmHWFVsbDNw1l1+4f90U6IY/q4V27dpnE9bJ+v87QEYdj qx/
UamVVPRG+mwkNTYN+9tjkwzEx+atCm/
X9WvWtDtAb68Wy9LXa1UmvCDDIpKy0Q5ZwSzJ4jMrvFcr0rSj0Uh+GcT4LSg5ugkW:
SCDQBojh0hPlaJdah+tkVYrnTZowP8iq1F1TgMBBauufyB33x1v+NWfYmT5KmpgHC-
UivS9NztpkVEDkCrs5alhhWP9NeqlfWopzhZScI6QxseegZRGeg5a8C3Re1Mfl1ScP:
FC3mp6jBhKK0iY2h3gtUV64TWM6wDETRPLDfSakXmH3w8g9Jlug8ZtTt4kVF0kLUYYr
DrQ5YhMGBA9L3ucdjh0y8k0HW5gU/VEEmJTcL4Pz/
f7mgoAbYkAAAAAElFTkSuQmCC"]
    }
  ]
}'

```

Response

```

{
  "model": "llava",
  "created_at": "2023-12-13T22:42:50.203334Z",
  "message": {
    "role": "assistant",
    "content": "The image features a cute, little pig with an angry facial expression. It's wearing a heart on its shirt and is waving in the air. This scene appears to be part of a drawing or sketching project.",
    "images": null
  },
  "done": true,
  "total_duration": 1668506709,
  "load_duration": 1986209,
  "prompt_eval_count": 26,
  "prompt_eval_duration": 359682000,
  "eval_count": 83,
  "eval_duration": 1303285000
}

```

Chat request (Reproducible outputs)

Request

```

curl http://localhost:11434/api/chat -d '{
  "model": "llama3.2",
  "messages": [
    {
      "role": "user",
      "content": "Hello!"
    }
  ]
}'

```

```
],
"options": {
  "seed": 101,
  "temperature": 0
}
}'
```

Response

```
{
  "model": "llama3.2",
  "created_at": "2023-12-12T14:13:43.416799Z",
  "message": {
    "role": "assistant",
    "content": "Hello! How are you today?"
  },
  "done": true,
  "total_duration": 5191566416,
  "load_duration": 2154458,
  "prompt_eval_count": 26,
  "prompt_eval_duration": 383809000,
  "eval_count": 298,
  "eval_duration": 4799921000
}
```

Chat request (with tools)

Request

```
curl http://localhost:11434/api/chat -d '{
  "model": "llama3.2",
  "messages": [
    {
      "role": "user",
      "content": "What is the weather today in Paris?"
    }
  ],
  "stream": false,
  "tools": [
    {
      "type": "function",
      "function": {
        "name": "get_current_weather",
        "description": "Get the current weather for a location",
        "parameters": {
          "type": "object",
          "properties": {
            "location": {
              "type": "string",
              "description": "The location to get the weather for,
e.g. San Francisco, CA"
            }
          }
        }
      }
    }
  ]
}
```

```

        "format": {
            "type": "string",
            "description": "The format to return the weather in,
e.g. 'celsius' or 'fahrenheit'",
            "enum": ["celsius", "fahrenheit"]
        },
        "required": ["location", "format"]
    }
}
]
}'

```

Response

```

{
  "model": "llama3.2",
  "created_at": "2024-07-22T20:33:28.123648Z",
  "message": {
    "role": "assistant",
    "content": "",
    "tool_calls": [
      {
        "function": {
          "name": "get_current_weather",
          "arguments": {
            "format": "celsius",
            "location": "Paris, FR"
          }
        }
      }
    ]
  },
  "done_reason": "stop",
  "done": true,
  "total_duration": 885095291,
  "load_duration": 3753500,
  "prompt_eval_count": 122,
  "prompt_eval_duration": 328493000,
  "eval_count": 33,
  "eval_duration": 552222000
}

```

Load a model

If the messages array is empty, the model will be loaded into memory.

Request

```
curl http://localhost:11434/api/chat -d '{
  "model": "llama3.2",
  "messages": []
}'
```

Response

```
{
  "model": "llama3.2",
  "created_at": "2024-09-12T21:17:29.110811Z",
  "message": {
    "role": "assistant",
    "content": ""
  },
  "done_reason": "load",
  "done": true
}
```

Unload a model

If the messages array is empty and the keep_alive parameter is set to 0, a model will be unloaded from memory.

Request

```
curl http://localhost:11434/api/chat -d '{
  "model": "llama3.2",
  "messages": [],
  "keep_alive": 0
}'
```

Response

A single JSON object is returned:

```
{
  "model": "llama3.2",
  "created_at": "2024-09-12T21:33:17.547535Z",
  "message": {
    "role": "assistant",
    "content": ""
  },
  "done_reason": "unload",
  "done": true
}
```

Create a Model

POST /api/create

Create a model from:

- another model;
- a safetensors directory; or
- a GGUF file.

If you are creating a model from a safetensors directory or from a GGUF file, you must create a blob for each of the files and then use the file name and SHA256 digest associated with each blob in the `files` field.

Parameters

- `model`: name of the model to create
- `from`: (optional) name of an existing model to create the new model from
- `files`: (optional) a dictionary of file names to SHA256 digests of blobs to create the model from
- `adapters`: (optional) a dictionary of file names to SHA256 digests of blobs for LORA adapters
- `template`: (optional) the prompt template for the model
- `license`: (optional) a string or list of strings containing the license or licenses for the model
- `system`: (optional) a string containing the system prompt for the model
- `parameters`: (optional) a dictionary of parameters for the model (see [Model file](#) for a list of parameters)
- `messages`: (optional) a list of message objects used to create a conversation
- `stream`: (optional) if `false` the response will be returned as a single response object, rather than a stream of objects
- `quantize` (optional): quantize a non-quantized (e.g. float16) model

Quantization types

Type	Recommended
q4_K_M	*
q4_K_S	
q8_0	*

Examples

Create a new model

Create a new model from an existing model.

Request

```
curl http://localhost:11434/api/create -d '{
  "model": "mario",
  "from": "llama3.2",
  "system": "You are Mario from Super Mario Bros."
}'
```

Response

A stream of JSON objects is returned:

```
{"status":"reading model metadata"}
{"status":"creating system layer"}
{"status":"using already created layer
  sha256:22f7f8ef5f4c791c1b03d7eb414399294764d7cc82c7e94aa81a1feb80a983a2"}
{"status":"using already created layer
  sha256:8c17c2ebb0ea011be9981cc3922db8ca8fa61e828c5d3f44cb6ae342bf80460b"}
{"status":"using already created layer
  sha256:7c23fb36d80141c4ab8cddb61ee4790102ebd2bf7aef414453177d4f2110e5d"}
{"status":"using already created layer
  sha256:2e0493f67d0c8c9c68a8aeacdf6a38a2151cb3c4c1d42accf296e19810527988"}
{"status":"using already created layer
  sha256:2759286baa875dc22de5394b4a925701b1896a7e3f8e53275c36f75a877a82c9"}
{"status":"writing layer
  sha256:df30045fe90f0d750db82a058109cecd6d4de9c90a3d75b19c09e5f64580bb42"}
{"status":"writing layer
  sha256:f18a68eb09bf925bb1b669490407c1b1251c5db98dc4d3d81f3088498ea55690"}
{"status":"writing manifest"}
{"status":"success"}
```

Quantize a model

Quantize a non-quantized model.

Request

```
curl http://localhost:11434/api/create -d '{
  "model": "llama3.2:quantized",
  "from": "llama3.2:3b-instruct-fp16",
  "quantize": "q4_K_M"
}'
```

Response

A stream of JSON objects is returned:

```
{"status":"quantizing F16 model to
  Q4_K_M","digest":"0","total":6433687776,"completed":12302}
{"status":"quantizing F16 model to
  Q4_K_M","digest":"0","total":6433687776,"completed":6433687552}
{"status":"verifying conversion"}
```

```

{"status":"creating new layer
  sha256:fb7f4f211b89c6c4928ff4ddb73db9f9c0cfca3e000c3e40d6cf27ddc6ca72eb"}
{"status":"using existing layer
  sha256:966de95ca8a62200913e3f8bfbf84c8494536f1b94b49166851e76644e966396"}
{"status":"using existing layer
  sha256:fcc5a6bec9daf9b561a68827b67ab6088e1dba9d1fa2a50d7bbcc8384e0a265d"}
{"status":"using existing layer
  sha256:a70ff7e570d97baaf4e62ac6e6ad9975e04caa6d900d3742d37698494479e0cd"}
{"status":"using existing layer
  sha256:56bb8bd477a519ffa694fc449c2413c6f0e1d3b1c88fa7e3c9d88d3ae49d4dcb"}
{"status":"writing manifest"}
{"status":"success"}

```

Create a model from GGUF

Create a model from a GGUF file. The files parameter should be filled out with the file name and SHA256 digest of the GGUF file you wish to use. Use [/api/blobs/:digest](#) to push the GGUF file to the server before calling this API.

Request

```

curl http://localhost:11434/api/create -d '{
  "model": "my-gguf-model",
  "files": {
    "test.gguf":
"sha256:432f310a77f4650a88d0fd59ecdd7cebed8d684bafea53cbff0473542964f0c3"
  }
}'

```

Response

A stream of JSON objects is returned:

```

{"status":"parsing GGUF"}
{"status":"using existing layer
  sha256:432f310a77f4650a88d0fd59ecdd7cebed8d684bafea53cbff0473542964f0c3"}
{"status":"writing manifest"}
{"status":"success"}

```

Create a model from a Safetensors directory

The files parameter should include a dictionary of files for the safetensors model which includes the file names and SHA256 digest of each file. Use [/api/blobs/:digest](#) to first push each of the files to the server before calling this API. Files will remain in the cache until the Ollama server is restarted.

Request

```

curl http://localhost:11434/api/create -d '{
  "model": "fred",
  "files": {

```

```

    "config.json":
    "sha256:dd3443e529fb2290423a0c65c2d633e67b419d273f170259e272972198:
    "generation_config.json":
    "sha256:88effbb63300dbbc7390143fbbdd9d9fa50587b37e8bfd16c8c90d4970:
    "special_tokens_map.json":
    "sha256:b7455f0e8f0539108837bfa586c4fbf424e31f8717819a6798be74bef8
    "tokenizer.json":
    "sha256:bbc1904d35169c542dffbe1f7589a5994ec7426d9e5b609d07bab876f3:
    "tokenizer_config.json":
    "sha256:24e8a6dc2547164b7002e3125f10b415105644fcf02bf9ad8b674c87b1e
    "model.safetensors":
    "sha256:1ff795ff6a07e6a68085d206fb84417da2f083f68391c2843cd2b8ac6d
  }
}'

```

Response

A stream of JSON objects is returned:

```

{"status":"converting model"}
{"status":"creating new layer
sha256:05ca5b813af4a53d2c2922933936e398958855c44ee534858fcfd8309406
{"status":"using autodetected template llama3-instruct"}
{"status":"using existing layer
sha256:56bb8bd477a519ffa694fc449c2413c6f0e1d3b1c88fa7e3c9d88d3ae49c
{"status":"writing manifest"}
{"status":"success"}

```

Check if a Blob Exists

HEAD /api/blobs/:digest

Ensures that the file blob (Binary Large Object) used with create a model exists on the server. This checks your Ollama server and not ollama.com.

Query Parameters

- digest: the SHA256 digest of the blob

Examples

Request

```

curl -I http://localhost:11434/api/blobs/
sha256:29fdb92e57cf0827ded04ae6461b5931d01fa595843f55d36f5b275a5208

```

Response

Return 200 OK if the blob exists, 404 Not Found if it does not.

Push a Blob

POST /api/blobs/:digest

Push a file to the Ollama server to create a “blob” (Binary Large Object).

Query Parameters

- digest: the expected SHA256 digest of the file

Examples

Request

```
curl -T model.gguf -X POST http://localhost:11434/api/blobs/  
sha256:29fdb92e57cf0827ded04ae6461b5931d01fa595843f55d36f5b275a520{
```

Response

Return 201 Created if the blob was successfully created, 400 Bad Request if the digest used is not expected.

List Local Models

GET /api/tags

List models that are available locally.

Examples

Request

```
curl http://localhost:11434/api/tags
```

Response

A single JSON object will be returned.

```
{  
  "models": [  
    {  
      "name": "deepseek-r1:latest",  
      "model": "deepseek-r1:latest",  
      "modified_at": "2025-05-10T08:06:48.639712648-07:00",  
      "size": 4683075271,  
      "digest":  
        "0a8c266910232fd3291e71e5ba1e058cc5af9d411192cf88b6d30e92b6e73163",  
      "details": {
```

```

        "parent_model": "",
        "format": "gguf",
        "family": "qwen2",
        "families": ["qwen2"],
        "parameter_size": "7.6B",
        "quantization_level": "Q4_K_M"
    },
    {
        "name": "llama3.2:latest",
        "model": "llama3.2:latest",
        "modified_at": "2025-05-04T17:37:44.706015396-07:00",
        "size": 2019393189,
        "digest":
            "a80c4f17acd55265feec403c7aef86be0c25983ab279d83f3bcd3abbcb5b8b72",
        "details": {
            "parent_model": "",
            "format": "gguf",
            "family": "llama",
            "families": ["llama"],
            "parameter_size": "3.2B",
            "quantization_level": "Q4_K_M"
        }
    }
]
}

```

Show Model Information

POST /api/show

Show information about a model including details, modelfile, template, parameters, license, system prompt.

Parameters

- `model`: name of the model to show
- `verbose`: (optional) if set to `true`, returns full data for verbose response fields

Examples

Request

```
curl http://localhost:11434/api/show -d '{
  "model": "llava"
}'
```

Response

```
{
  modelfile: '# Modelfile generated by "ollama show"\n# To build a
new Modelfile based on this one, replace the FROM line with:\n#
FROM llava:latest\n\nFROM /Users/matt/.ollama/models/blobs/
sha256:200765e1283640ffbd013184bf496e261032fa75b99498a9613be4e94d6:
""""{{ .System }}\nUSER: {{ .Prompt }}\nASSISTANT: """"\nPARAMETER
num_ctx 4096\nPARAMETER stop "\u003c/s\u003e"\nPARAMETER stop
"USER:"\nPARAMETER stop "ASSISTANT:"',
  parameters: 'num_keep
24\nstop                                "<|start_header_id|
>"\nstop                                "<|end_header_id|
>"\nstop                                "<|eot_id|>"',
  template: '{{ if .System }}<|start_header_id|>system<|
end_header_id|>\n\n{{ .System }}<|eot_id|>{{ end }}
{{ if .Prompt }}<|start_header_id|>user<|end_header_id|
>\n\n{{ .Prompt }}<|eot_id|>{{ end }}<|start_header_id|
>assistant<|end_header_id|>\n\n{{ .Response }}<|eot_id|>',
  details: {
    parent_model: '',
    format: "gguf",
    family: "llama",
    families: ["llama"],
    parameter_size: "8.0B",
    quantization_level: "Q4_0",
  },
  model_info: {
    "general.architecture": "llama",
    "general.file_type": 2,
    "general.parameter_count": 8030261248,
    "general.quantization_version": 2,
    "llama.attention.head_count": 32,
    "llama.attention.head_count_kv": 8,
    "llama.attention.layer_norm_rms_epsilon": 0.00001,
    "llama.block_count": 32,
    "llama.context_length": 8192,
    "llama.embedding_length": 4096,
    "llama.feed_forward_length": 14336,
    "llama.rope.dimension_count": 128,
    "llama.rope.freq_base": 500000,
    "llama.vocab_size": 128256,
    "tokenizer.ggml.bos_token_id": 128000,
    "tokenizer.ggml.eos_token_id": 128009,
    "tokenizer.ggml.merges": [], // populates if `verbose=true`
    "tokenizer.ggml.model": "gpt2",
    "tokenizer.ggml.pre": "llama-bpe",
    "tokenizer.ggml.token_type": [], // populates if
`verbose=true`
    "tokenizer.ggml.tokens": [], // populates if `verbose=true`
  },
}
```



```
  capabilities: ["completion", "vision"],
}
```

Copy a Model

POST /api/copy

Copy a model. Creates a model with another name from an existing model.

Examples

Request

```
curl http://localhost:11434/api/copy -d '{
  "source": "llama3.2",
  "destination": "llama3-backup"
}'
```

Response

Returns a 200 OK if successful, or a 404 Not Found if the source model doesn't exist.

Delete a Model

DELETE /api/delete

Delete a model and its data.

Parameters

- model: model name to delete

Examples

Request

```
curl -X DELETE http://localhost:11434/api/delete -d '{
  "model": "llama3:13b"
}'
```

Response

Returns a 200 OK if successful, 404 Not Found if the model to be deleted doesn't exist.

Pull a Model

POST /api/pull

Download a model from the ollama library. Cancelled pulls are resumed from where they left off, and multiple calls will share the same download progress.

Parameters

- `model`: name of the model to pull
- `insecure`: (optional) allow insecure connections to the library. Only use this if you are pulling from your own library during development.
- `stream`: (optional) if `false` the response will be returned as a single response object, rather than a stream of objects

Examples

Request

```
curl http://localhost:11434/api/pull -d '{
  "model": "llama3.2"
}'
```

Response

If `stream` is not specified, or set to `true`, a stream of JSON objects is returned:

The first object is the manifest:

```
{
  "status": "pulling manifest"
}
```

Then there is a series of downloading responses. Until any of the download is completed, the `completed` key may not be included. The number of files to be downloaded depends on the number of layers specified in the manifest.

```
{
  "status": "pulling digestname",
  "digest": "digestname",
  "total": 2142590208,
  "completed": 241970
}
```

After all the files are downloaded, the final responses are:

```
{
  "status": "verifying sha256 digest"
}
{
  "status": "writing manifest"
}
```

```
{
  "status": "removing any unused layers"
}
{
  "status": "success"
}
```

if `stream` is set to `false`, then the response is a single JSON object:

```
{
  "status": "success"
}
```

Push a Model

POST `/api/push`

Upload a model to a model library. Requires registering for `ollama.ai` and adding a public key first.

Parameters

- `model`: name of the model to push in the form of `<namespace>/<model>:<tag>`
- `insecure`: (optional) allow insecure connections to the library. Only use this if you are pushing to your library during development.
- `stream`: (optional) if `false` the response will be returned as a single response object, rather than a stream of objects

Examples

Request

```
curl http://localhost:11434/api/push -d '{
  "model": "mattw/pygmalion:latest"
}'
```

Response

If `stream` is not specified, or set to `true`, a stream of JSON objects is returned:

```
{ "status": "retrieving manifest" }
```

and then:

```
{
  "status": "starting upload",
  "digest":
    "sha256:bc07c81de745696fdf5afca05e065818a8149fb0c77266fb584d9b2cba3711ab"
```

```
"total": 1928429856
}
```

Then there is a series of uploading responses:

```
{
  "status": "starting upload",
  "digest":
    "sha256:bc07c81de745696fdf5afca05e065818a8149fb0c77266fb584d9b2cba3711ab",
  "total": 1928429856
}
```

Finally, when the upload is complete:

```
{"status":"pushing manifest"}
{"status":"success"}
```

If stream is set to false, then the response is a single JSON object:

```
{ "status": "success" }
```

Generate Embeddings

POST /api/embed

Generate embeddings from a model

Parameters

- model: name of model to generate embeddings from
- input: text or list of text to generate embeddings for

Advanced parameters:

- truncate: truncates the end of each input to fit within context length. Returns error if false and context length is exceeded. Defaults to true
- options: additional model parameters listed in the documentation for the [Modelfile](#) such as temperature
- keep_alive: controls how long the model will stay loaded into memory following the request (default: 5m)
- dimensions: number of dimensions for the embedding

Examples

Request

```
curl http://localhost:11434/api/embed -d '{
  "model": "all-minilm",
  "input": "Why is the sky blue?"
}'
```

Response

```
{
  "model": "all-minilm",
  "embeddings": [
    [
      0.010071029, -0.0017594862, 0.05007221, 0.04692972,
      0.054916814,
      0.008599704, 0.105441414, -0.025878139, 0.12958129,
      0.031952348
    ]
  ],
  "total_duration": 14143917,
  "load_duration": 1019500,
  "prompt_eval_count": 8
}
```

Request (Multiple input)

```
curl http://localhost:11434/api/embed -d '{
  "model": "all-minilm",
  "input": ["Why is the sky blue?", "Why is the grass green?"]
}'
```

Response

```
{
  "model": "all-minilm",
  "embeddings": [
    [
      0.010071029, -0.0017594862, 0.05007221, 0.04692972,
      0.054916814,
      0.008599704, 0.105441414, -0.025878139, 0.12958129,
      0.031952348
    ],
    [
      -0.0098027075, 0.06042469, 0.025257962, -0.006364387,
      0.07272725,
      0.017194884, 0.09032035, -0.051705178, 0.09951512,
      0.09072481
    ]
  ]
}
```

List Running Models

GET /api/ps

List models that are currently loaded into memory.

Examples

Request

```
curl http://localhost:11434/api/ps
```

Response

A single JSON object will be returned.

```
{
  "models": [
    {
      "name": "mistral:latest",
      "model": "mistral:latest",
      "size": 5137025024,
      "digest":
        "2ae6f6dd7a3dd734790bbbf58b8909a606e0e7e97e94b7604e0aa7ae4490e6d8",
      "details": {
        "parent_model": "",
        "format": "gguf",
        "family": "llama",
        "families": ["llama"],
        "parameter_size": "7.2B",
        "quantization_level": "Q4_0"
      },
      "expires_at": "2024-06-04T14:38:31.83753-07:00",
      "size_vram": 5137025024
    }
  ]
}
```

Generate Embedding

| Note: this endpoint has been superseded by /api/embed

POST /api/embeddings

Generate embeddings from a model

Parameters

- model: name of model to generate embeddings from
- prompt: text to generate embeddings for

Advanced parameters:

- options: additional model parameters listed in the documentation for the Modelfile such as temperature

- `keep_alive`: controls how long the model will stay loaded into memory following the request (default: 5m)

Examples

Request

```
curl http://localhost:11434/api/embeddings -d '{
  "model": "all-minilm",
  "prompt": "Here is an article about llamas..."
}'
```

Response

```
{
  "embedding": [
    0.5670403838157654, 0.009260174818336964, 0.23178744316101074,
    -0.2916173040866852, -0.8924556970596313, 0.8785552978515625,
    -0.34576427936553955, 0.5742510557174683,
    -0.04222835972905159,
    -0.137906014919281
  ]
}
```

Version

GET `/api/version`

Retrieve the Ollama version

Examples

Request

```
curl http://localhost:11434/api/version
```

Response

```
{
  "version": "0.5.1"
}
```

Experimental Features

Image Generation (Experimental)

[!WARNING] Image generation is experimental and may change in future versions.

Image generation is now supported through the standard `/api/generate` endpoint when using image generation models. The API automatically detects when an image generation model is being used.

See the [Generate a completion](#) section for the full API documentation. The experimental image generation parameters (width, height, steps) are documented there.

Example

Request

```
curl http://localhost:11434/api/generate -d '{
  "model": "x/z-image-turbo",
  "prompt": "a sunset over mountains",
  "width": 1024,
  "height": 768
}'
```

Response (streaming)

Progress updates during generation:

```
{
  "model": "x/z-image-turbo",
  "created_at": "2024-01-15T10:30:00.000000Z",
  "completed": 5,
  "total": 20,
  "done": false
}
```

Final Response

```
{
  "model": "x/z-image-turbo",
  "created_at": "2024-01-15T10:30:15.000000Z",
  "image": "iVBORw0KGgoAAAANSUhEUg...",
  "done": true,
  "done_reason": "stop",
  "total_duration": 15000000000,
  "load_duration": 2000000000
}
```